VC HW1 report

Ablation experiments:

All ablation experiments were evaluated using lena_gray.png. BPP was calculated from the actual .bin file size, including the 4-byte header and final padding, as required by the assignment. The temporary ablation-testing harness was not included in the final submission.

1. Custom modifications to the Quantization table

   In this experiment, every entry in the standard quantization table was **multiplied by 2** to investigate the effect of a larger quantization step on compression efficiency and image quality.

Table 1. The evaluation table of Quantization table x2 and the baseline.

Configuration	MSE	PSNR (dB)	Payload bits	BPP
Standard table	17.0697	35.8085	219548	0.8376
Quant. table x2	27.7068	33.7049	149952	0.5721

Increasing the quantization values makes the quantization steps larger, causing **more** transform **information** to be **discarded**. More quantized coefficients become zero, while the remaining nonzero coefficients generally have smaller magnitudes. This **improves entropy-coding efficiency, reducing the payload size and BPP**.

However, the coarser quantization introduces greater reconstruction error, resulting in a higher MSE and a lower PSNR. Compared with the baseline, doubling the quantization table reduces BPP by approximately **31.7%**, while PSNR decreases by approximately **2.10 dB**. These results demonstrate the expected trade-off between compression efficiency and reconstruction quality.

2. Zigzag Scan vs. Raster Scan (BPP and PSNR)

   For the raster experiment, coefficients were traversed in **ordinary row-major order**. The same scanning order was used during encoding and decoding to ensure that the coefficients were reconstructed correctly.

Table 2. The evaluation table of Zigzag and Raster scan.

Configuration	MSE	PSNR (dB)	Payload bits	BPP
Zigzag scan	17.0697	35.8085	219548	0.8376
Raster scan	17.0697	35.8085	265342	1.0123

The MSE and PSNR remain identical because changing the scan order does not change the quantized coefficient values. It **only changes** their serialization and, consequently, their **entropy-coding efficiency**.

Zigzag scanning arranges the coefficients approximately from **lower to higher spatial frequencies**. Since quantization tends to eliminate high-frequency coefficients, the resulting sequence often contains a **long trailing run of zeros**, which can be **efficiently represented** using the End-of-Block (EOB) symbol.

In contrast, raster scanning does not preserve this frequency ordering. **Nonzero coefficients may appear later** in the sequence, **reducing** the **effectiveness** of run-length coding and EOB termination. Consequently, raster scanning increases BPP by approximately **20.9%** without improving reconstruction quality. This demonstrates why zigzag scanning is beneficial for JPEG entropy coding.

3. Two stage 1D DCT vs. Direct 2D DCT (Time, bpp)

The direct implementation used the required double-summation equation:

$$F(u,v) = \sum_{x=0}^{7}\sum_{y=0}^{7} C(u,x)f(u,x)C(v,y)$$

Timing measured only DCT computation over 8192 blocks (two passes over the 4096 blocks in the image). Each implementation was measured **11 times** using Python's `time.perf_counter()`. The **first** measurement was **discarded** to reduce the influence of **startup overhead**, and the mean of the remaining 10 measurements was reported.

PSNR and BPP are reported to six decimal places because the differences between the two implementations are small.

Table 3. The evaluation table of Two stage 1D DCT vs. Direct 2D DCT.

Configuration	DCT time (s)	PSNR (dB)	Payload bits	BPP
Two-stage 1D DCT	0.044471	35.808538	219548	0.837646
Direct 2D DCT	8.084186	35.808498	219544	0.837616

The direct implementation took approximately **181.8 times longer** than the two-stage implementation in this Python experiment. This difference is primarily attributable to computational complexity and implementation overhead.

For an $N \times N$ block, the direct 2D DCT computes N^2 output coefficients,

each requiring a double summation over N^2 input samples. Therefore, its computational complexity is $O(N^4)$. For an 8×8 block, this corresponds to **4,096** summation terms.

The 2D DCT is separable and can instead be computed using two 1D DCT passes, one along each dimension. Each pass requires $O(N^3)$. operations, giving a total complexity of 8×8. For an 8×8 block, two matrix multiplications require approximately **1,024** multiply-accumulate terms, compared with 4,096 terms for the direct implementation. Thus, the theoretical ratio of these term counts is approximately **4:1**.

In this experiment, however, the measured speedup was approximately **181.8** times. This reflects not only the **reduction in arithmetic operations** but also the **implementation differences**: the direct method uses Python-level nested loops, whereas the two-stage method uses **matrix multiplication**, which is implemented in **optimized numerical libraries**.

The two implementations also produce nearly identical PSNR and BPP values. The small differences are likely caused by **floating-point rounding** near quantization thresholds, which can change a few quantized coefficients and their encoded representation. These differences are **negligible** in terms of reconstruction quality and compression efficiency. Overall, the results demonstrate that the **separable two-stage DCT** achieves substantially **better computational efficiency** while **preserving** the expected **numerical behavior** of the direct 2D DCT.